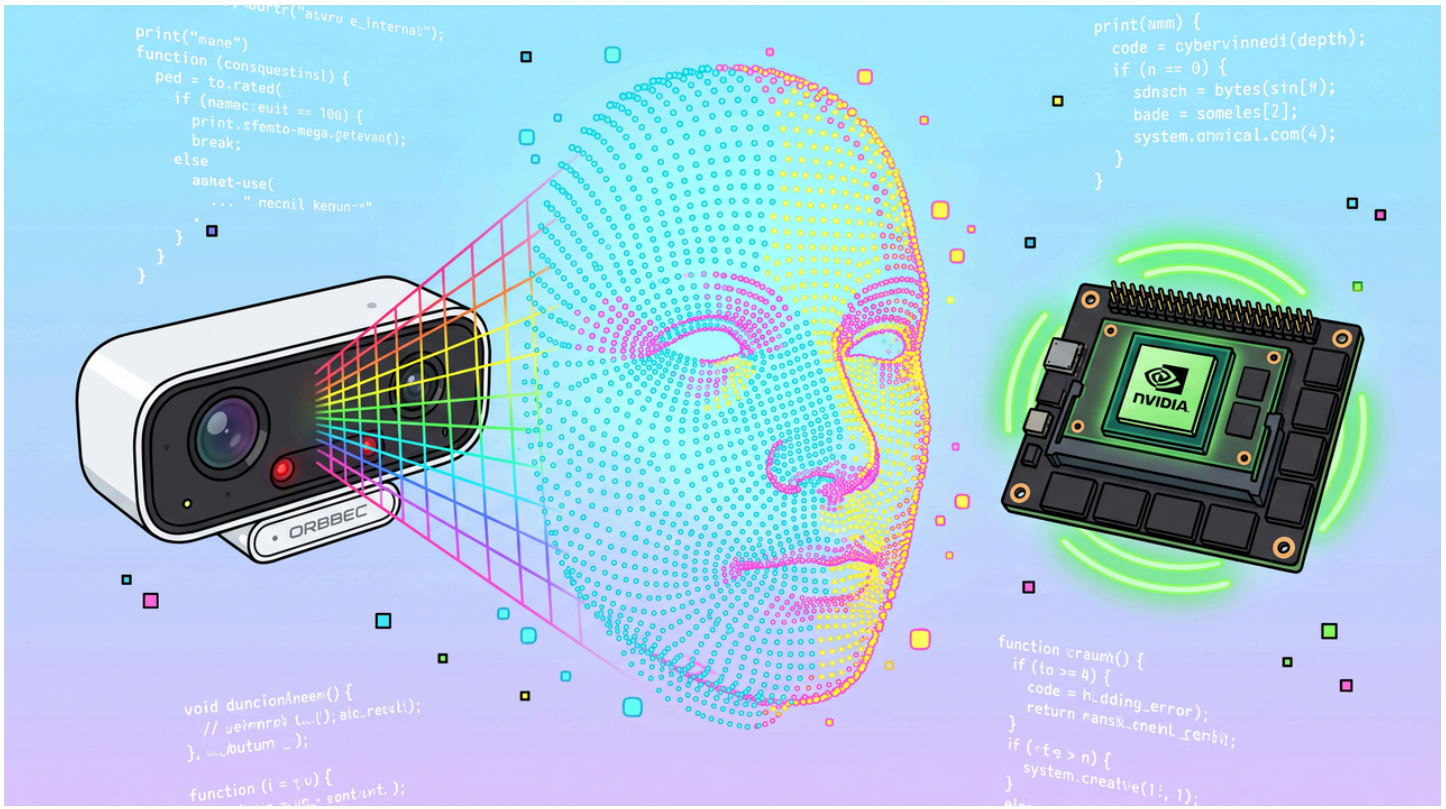
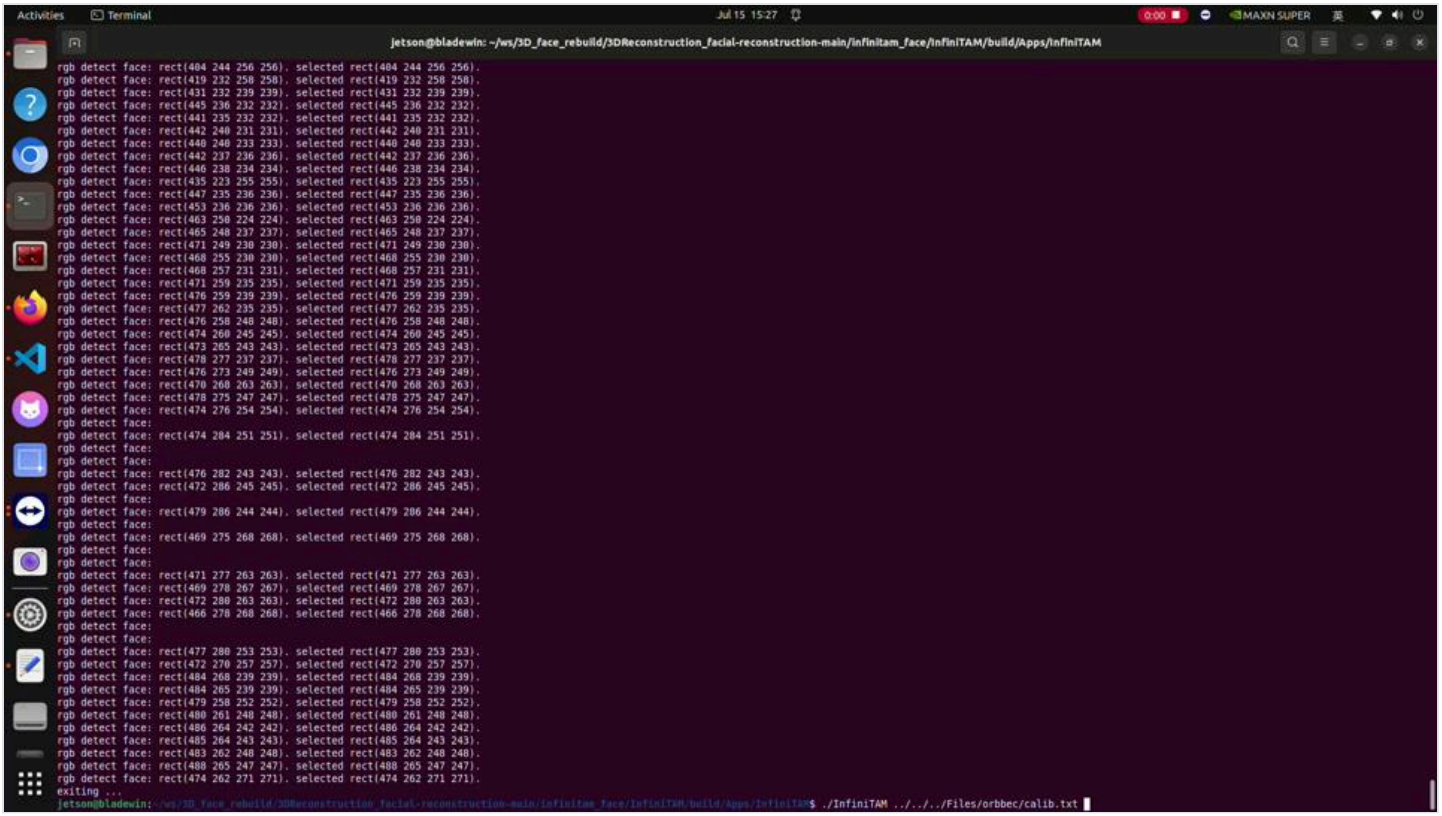
Jetson Orin Nano + Orbbec Femto Mega 复现 3D 人脸重建项目教程 🚀



> 🖥️适用平台：Jetson Orin Nano / Ubuntu 22.04 / Orbbec Femto Mega / OrbbecSDK v2 / InfiniTAM Face Reconstruction
>
> 🔗GitHub



1. 这次我们解决了什么？✅

原项目默认优先使用 OpenNI / RealSense 输入源，但 Orbbec Femto Mega 在 Jetson 上通常不会通过系统自带 OpenNI 驱动暴露出来，所以会出现：

代码块

```
OpenNI: Device open failed!
DeviceOpen using default: no devices found
failed to open any image stream
```

这次改造把输入引擎迁移到了 **OrbbecSDK v2**，让程序直接通过 Orbbec 官方 SDK 读取 Femto Mega 的 RGB-D 数据。

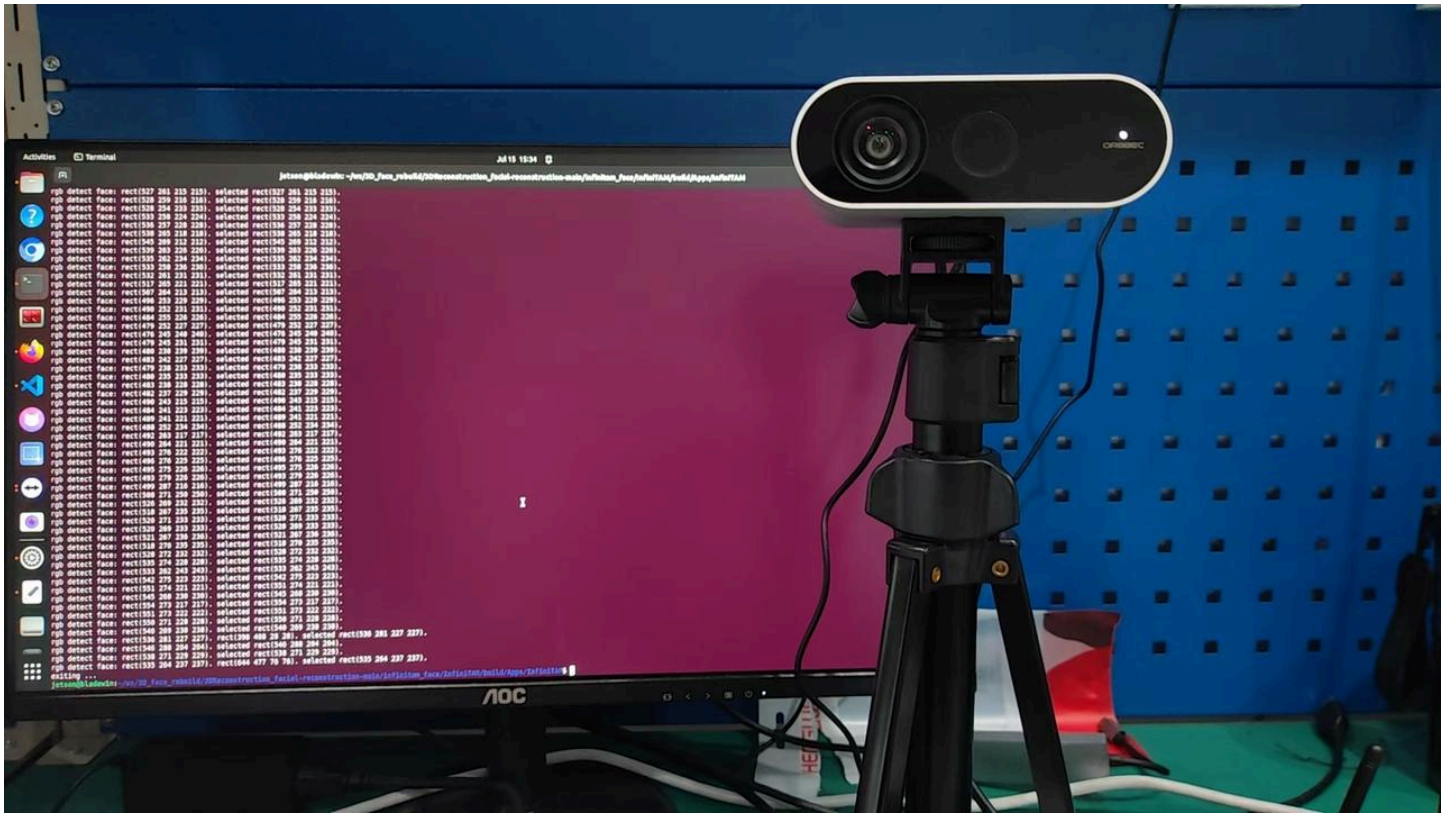
同时还做了 3 个关键修复：

- ✨ 使用 OrbbecSDK v2 获取 RGB / Depth 帧
- 🎯 启用硬件 depth-to-color 对齐
- 📐 通过 SDK 获取真实相机内参，并写入 InfiniTAM

- 🧹 人脸检测阶段过滤小误检，并选择最大面积的人脸框

2. 硬件与软件环境

推荐环境如下：



代码块

```
硬件：Jetson Orin Nano
系统：Ubuntu 22.04
相机：Orbbec Femto Mega
SDK：OrbbecSDK v2，例如 2.8.6
CUDA Compute Capability：8.7
```

检查相机是否被 USB 识别：

代码块

```
lsusb
```

看到类似下面的设备说明 USB 层面 OK：

```
代码块
1   2bc5:0669 Orbbec 3D Technology International, Inc Orbbec Femto Mega 3D Camera
```

检查 OrbbecSDK 是否安装：

```
代码块
1   ls /usr/local/lib | grep Orbbec
```

正常应看到：

```
代码块
1   libOrbbecSDK.so
2   libOrbbecSDK.so.2
3   libOrbbecSDK.so.2.8.6
```

检查头文件：

```
代码块
1   ls /usr/local/include/libobsensor/ObSensor.hpp
```

3. 工程关键改造点 🛠️

3.1 新增 OrbbecSDK2 输入源

新增文件：

```
代码块
1   InfiniTAM/InputSource/OrbbecSDK2Engine.h
2   InfiniTAM/InputSource/OrbbecSDK2Engine.cpp
```

核心逻辑：

```
代码块
1   pipe = std::make_shared<ob::Pipeline>();
2   pipe->enableFrameSync();
3
4   auto colorProfiles = pipe->getStreamProfileList(OB_SENSOR_COLOR);
5   auto depthProfiles = pipe->getStreamProfileList(OB_SENSOR_DEPTH);
6
7   config->enableStream(colorProfile);
8   config->enableStream(depthProfile);
9   config->setAlignMode(ALIGN_D2C_HW_MODE);
10
11  pipe->start(config);
```

程序启动后会优先尝试：

```
代码块
1   trying Orbbec device with SDK v2
```

3.2 使用 SDK 真实内参

原来的 `Files/orbbec/calib.txt` 是 640x480 旧参数，而 Femto Mega 实际运行时可能是：

```
代码块
1   color 1280x720
2   depth 1280x720
```

所以必须用 SDK 读取当前 profile 对应的真实内参：

```
代码块
1   const OBCameraParam cameraParam = pipe->getCameraParam();
2   setInfiniTAMIntrinsics(calib.intrinsics_rgb, cameraParam.rgbIntrinsic,
    imageSize_rgb);
```

由于启用了 depth-to-color 对齐，depth 已经处在 color 坐标系下，因此 depth 内参也使用 color 内参：

```
代码块
1   setInfiniTAMIntrinsics(calib.intrinsics_d, cameraParam.rgbIntrinsic,
    imageSize_d);
2   calib.trafo_rgb_to_depth = ITMLib::ITMExtrinsics();
```

正常运行时会看到：

```
代码块
1   OrbbecSDK2: using SDK color intrinsics for aligned RGB-D stream fx=745.446
    fy=745.367 cx=634.326 cy=351.723
```

3.3 人脸框筛选优化

原代码要求“刚好检测到 1 张脸”才继续处理：

代码块

```
if (ccRects.size() != 1) {
    return;
}
```

这会导致很多帧被丢弃。现在改为：

- 先过滤太小的误检框
- 再选择面积最大的人脸框
- 只把这个框用于深度裁剪和后续重建

核心代码：

代码块

```
const int minFaceSide = 120;
cv::Rect selectedFace;
int selectedArea = 0;

for (int i = 0; i < rects.size(); i++) {
    const cv::Rect& r = rects[i];
    if (r.width < minFaceSide || r.height < minFaceSide) continue;

    const int area = r.width * r.height;
    if (area > selectedArea) {
        selectedFace = r;
        selectedArea = area;
    }
}

ccRects.clear();
if (selectedArea > 0) {
    ccRects.push_back(selectedFace);
```

```
19  }
```

运行时会看到：

代码块

```
1  rgb detect face: rect(...). rect(...). selected rect(...).
```

4. CMake 配置方式 ⚙️

进入工程：

代码块

```
1  cd ~/ws/3D_face_rebuild/3DReconstruction_facial-reconstruction-
   main/infinitam_face/InfiniTAM
```

建议每次大改后干净构建：

代码块

```
1  rm -rf build
2  mkdir build
3  cd build
```

配置 CMake：

代码块

```
1  cmake .. \
```

```
    -DWITH_CUDA=ON \
    -DCUDA_COMPUTE_CAPABILITY=87 \
    -DWITH_ORBBECSDK2=ON \
    -DORBBECSDK2_INCLUDE_DIR=/usr/local/include \
    -DORBBECSDK2_LIBRARY=/usr/local/lib/libOrbbecSDK.so \
    -DWITH_OPENNI=OFF \
    -DWITH_REALSENSE=OFF \
    -DWITH_REALSENSE2=OFF \
    -DWITH_FFMPEG=OFF
```

编译：

代码块

```
make -j$(nproc)
```

5. 运行方式 🎬

编译完成后进入可执行文件目录：

代码块

```
cd ~/ws/3D_face_rebuild/3DReconstruction_facial-reconstruction-main/infinitam_face/InfiniTAM/build/Apps/InfiniTAM
```

设置运行时库路径：

代码块

```
export LD_LIBRARY_PATH=/usr/local/lib:$LD_LIBRARY_PATH
```

启动程序：

代码块

```
./InfiniTAM ../../../Files/orbbec/calib.txt
```

看到下面几行说明 OrbbecSDK2 输入链路已经跑通：

代码块

```
trying Orbbec device with SDK v2
OrbbecSDK2: hardware depth-to-color alignment enabled
OrbbecSDK2: color profile 1280x720 fps=30 format=22
OrbbecSDK2: depth profile 640x576 fps=30 format=8
OrbbecSDK2: using SDK color intrinsics for aligned RGB-D stream fx=...
OrbbecSDK2: initialized color 1280x720, depth 1280x720
initialised.
```

窗口打开后：

- 按 `b`：连续处理视频流
- 按 `n`：只处理一帧
- 按 `f`：切换 free viewpoint / follow camera
- 按 `c`：切换显示模式
- 按 `e` 或 `Esc`：退出

6. 为什么有时很难看到 3D 人脸？🤔

这不是单纯的相机问题，而是这类 SLAM/TSDF 重建对输入质量要求比较高。

常见原因：

6.1 人脸检测不稳定

当前使用的是 OpenCV Haar Cascade：

```
代码块
1   haarcascade_frontalface_alt2.xml
```

它容易出现：

- 漏检
- 小框误检
- 多框误检
- 框在脸上跳来跳去

如果没有稳定选中人脸，后面的 depth crop 也会跟着乱跳。

6.2 代码会裁掉人脸框外的所有深度

程序会执行：

```
代码块
1   ClipDepthImage(inputRawDepthImage, inputRGBImage->noDims, ccRects[0]);
```

也就是只保留人脸框附近的深度，其余深度全部清零。

好处是背景不会进入重建；坏处是框稍微偏一点，脸部深度就会被裁掉。

6.3 InfiniTAM 不是单帧点云显示器

InfiniTAM 是 TSDF 融合系统，需要连续、稳定、有位姿变化的 RGB-D 帧。

如果：

- 人脸太小
- 人脸框跳动
- 相机晃动太快
- 被拍的人动太快
- 深度质量不稳定

那么 tracking 很容易 poor / failure，模型就不容易长出来。

7. 推荐采集姿势 📸

为了更容易看到重建结果，建议这样操作：

- 人脸距离相机约 40-80 cm
- 正脸面对相机
- 光照均匀，不要背光
- 人和相机都尽量慢慢移动
- 保证日志里持续出现 `selected rect(...)`
- 尽量让 selected rect 宽高大于 180 像素

比较理想的日志：

代码块

```
rgb detect face: rect(410 300 230 230). selected rect(410 300 230 230).
```

不太理想的日志：

代码块

```
rgb detect face:
rgb detect face: rect(954 571 45 45).
rgb detect face: rect(382 331 256 256). rect(564 358 112 112).
```

8. 常见报错与解决 🧯

8.1 `cannot find -lrealsense2`

原因：原 CMake 里有无条件链接 RealSense2。

解决：关闭 RealSense2，并移除无条件链接：

代码块

```
-DWITH_REALSENSE2=OFF
```

8.2 `OpenNI.h: No such file or directory`

原因： `WITH_OPENNI=OFF` 后， `OpenNIEngine.cpp` 仍然无条件 include OpenNI 头文件。

解决：让 OpenNIEngine 根据 `COMPILE_WITHOUT_OpenNI` 编译空壳实现。

8.3 `OPENNI_ROOT-NOTFOUND/Redist/OpenNI2`

原因：OpenNI 拷贝运行时目录时没有判断路径是否存在。

解决：`LinkOpenNI.cmake` 中对 `${OPENNI_ROOT}/Redist/OpenNI2` 加 `EXISTS` 判断。

8.4 `undefined reference to OrbbecSDK2Engine`

原因：`OrbbecSDK2Engine.cpp` 没有进入最终可执行文件链接。

解决：确保下面两个 CMake 文件都包含 Orbbec 源文件：

代码块

```
1   InputSource/CMakeLists.txt
2   Apps/InfiniTAM/CMakeLists.txt
```

8.5 `invalid static_cast from uint8_t* to uint16_t*`

原因：OrbbecSDK 的 `getData()` 返回字节指针。

解决：

代码块

```
1   const uint16_t *src = reinterpret_cast<const uint16_t *>(depthFrame-
    >getData());
```

9. 下一步可优化方向 🌱

当前版本已经可以跑通 Orbbec Femto Mega 输入，但如果想让人脸重建更稳，可以继续优化：

- 用 MediaPipe / YOLO-face 替换 Haar Cascade
- 对人脸框做时间滤波，减少跳动
- 放宽 depth crop 区域，避免裁掉脸颊和下巴
- 保存 selected RGB-D 帧，离线检查深度是否干净
- 增加 tracking 状态日志，判断是否真的融合成功
- 做一个单帧点云预览模式，先确认 Femto Mega 深度质量

10. 一句话总结 🎉

这次复现的核心不是“装一个 OpenNI 驱动”，而是绕开 OpenNI，直接使用 **OrbbecSDK v2 + SDK 内参 + D2C 对齐** 接入 InfiniTAM。

当日志出现：

```
代码块
1   OrbbecSDK2: using SDK color intrinsics for aligned RGB-D stream
2   selected rect(...)
```

说明输入链路和人脸筛选已经进入正确轨道啦！